Algorithm Complexity & Recurrence Relations

Complete Answer Key — GATE-Style Questions (Q1–Q15)

Q1. $T(n) = T(n/2) + T(2n/5) + 7n$, $T(0) = 1$

✓ **Answer: $T(n) = \Theta(n)$**

Q2. $T(n) = aT(n/b) + f(n)$, for constants $a \geq 1$, $b > 1$. When is $T(n) = \Theta(n \log n)$?

By the Master Theorem, compare $f(n)$ with $n^{\log_b a}$:

$T(n) = \Theta(n \log n)$ corresponds to **Case 2** of the Master Theorem, which requires: $a = b$ (so that $\log a = 1$, i.e. $n^{\log_b a} = n$)

and $f(n) = \Theta(n)$ (i.e. $f(n) = \Theta(n^{\log_b a} \cdot \log^0 n)$, with $k = 0$)

✓ **Answer: Condition: $a = b$ and $f(n) = \Theta(n)$**

Example: $T(n) = 2T(n/2) + n$ (merge sort) gives $a=2$, $b=2$ (so $\log a = 1$) and $f(n)=\Theta(n)$, yielding $T(n) = \Theta(n \log n)$ — exactly this case.

Q3. An algorithm checks if an array of size N is sorted via a single pass comparing only adjacent elements. Tightest running time?

A single pass compares each pair of adjacent elements exactly once: (a[0],a[1]), (a[1],a[2]), ..., (a[N-2], a[N-1]) — that's N−1 comparisons, regardless of whether the array is sorted, reverse sorted, or anything else. There is no early benefit that changes the asymptotic count in the best/average/worst case for a true "single pass that must inspect all adjacent pairs to certify sortedness" (even with early exit on the first violation, worst case still requires N−1 checks when the array is sorted).

✓ **Answer: $\Theta(N)$**

Q4. Evaluate the asymptotic order of $1^3 + 2^3 + ... + n^3$. Which are true: $\Theta(n^4)$, $\Theta(n^5)$, $O(n^5)$, $\Omega(n^3)$?

The closed form is the well-known sum of cubes:

$1^3 + 2^3 + ... + n^3 = [n(n+1)/2]^2 = \Theta(n^4)$

Now evaluate each option against $\Theta(n^4)$:

Option	Verdict	Reason
$\Theta(n^4)$	TRUE	Matches the exact closed form.
$\Theta(n^5)$	FALSE	Θ requires a tight bound; n^5 is not tight (grows strictly faster).
$O(n^5)$	TRUE	Any upper bound looser than the tight one still validly upper-bounds it.
$\Omega(n^3)$	TRUE	n^4 grows faster than n^3, so it is indeed lower-bounded by n^3.

✓ **Answer: $\Theta(n^4)$ — TRUE, $\Theta(n^5)$ — FALSE, $O(n^5)$ — TRUE, $\Omega(n^3)$ — TRUE**

Q5. fun1(n): Find the asymptotic value of q (nested loop with halving inner loop).

```
int fun1(int n)
{
int i,j,k,p,q=0;
for(i=1;i<n;++i)
{
p=0;
for(j=n;j>1;j=j/2)
++p;
for(k=1;k<p;k=k*2)
++q;
}
return q;
}
```

Step 1 — inner loop on j: j starts at n and is halved each iteration until j ≤ 1. This runs log n times, so p

2

= Θ(log n) after this loop.

Step 2 — loop on k: k starts at 1 and doubles each iteration while k < p. Since p = Θ(log n), this loop runs Θ(log p) = Θ(log log n) times, and q is incremented once per iteration.

Step 3 — outer loop on i: runs Θ(n) times, and each time it adds Θ(log log n) to q.

Total: q = Θ(n log log n)

✓ **Answer: q = $\Theta(n \cdot \log(\log n))$**

Q6. for(i=n; i>0; i/=2) { for(j=0; j<i; j++) { ... } }

```
for(int i=n;i>0;i/=2)
{
for(int j=0;j<i;j++)
{
// constant work
}
}
```

The outer loop runs with i = n, n/2, n/4, n/8, ..., 1 — that is Θ(log n) iterations. For each value of i, the inner loop runs exactly i times. So total work is:

n + n/2 + n/4 + ... + 1 = n(1 + 1/2 + 1/4 + ...) ≤ 2n → this is a geometric series that sums to Θ(n).

✓ **Answer: $\Theta(n)$**

Q7. for(i=1; i<=n; i++) { for(j=1; j<=i; j++) { ... } }

```
for(int i=1;i<=n;i++)
{
for(int j=1;j<=i;j++)
{
// constant work
}
}
```

The inner loop runs i times for each i. Total iterations across all i from 1 to n:

$1 + 2 + 3 + ... + n = n(n+1)/2 = \Theta(n^2)$

✓ **Answer: $\Theta(n^2)$**

Q8. fun(n): if(n≤1) return; fun(n/2); fun(n/2);

```
void fun(int n)
{
if(n<=1)
return;
fun(n/2);
fun(n/2);
}
```

Recurrence: $T(n) = 2T(n/2) + \Theta(1)$. By the Master Theorem with a=2, b=2, $f(n)=\Theta(1)$: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$, and $f(n) = O(n^{1-\varepsilon})$ for any $\varepsilon > 0$ (since $f(n) = \Theta(1)$ is polynomially smaller than n) — this is **Case 1**.

✓ **Answer: $T(n) = \Theta(n)$**

Q9. fib(n): if(n≤1) return n; return fib(n-1)+fib(n-2);

```
int fib(int n)
{
if(n<=1)
return n;
return fib(n-1)+fib(n-2);
}
```

✓ **Answer: $T(n) = O(2^n)$**

Q10. Arrange in increasing order: $n^{3/2}$, $n \log n$, $n^{\log_2 n}$, 2^n

✓ **Answer: $n \log n < n^{3/2} < n^{\log_2 n} < 2^n$**

Q11. T(n) = T(n-1) + 1

Unrolling: $T(n) = T(n-1) + 1 = T(n-2) + 2 = ... = T(0) + n$. There are n levels of recursion, each contributing constant work.

✓ **Answer: T(n) = Θ(n)**

Q12. T(n) = T(n-1) + n

Unrolling: $T(n) = T(n-1) + n = T(n-2) + (n-1) + n = ... = T(0) + (1+2+...+n) = T(0) + n(n+1)/2$.

✓ **Answer: T(n) = Θ(n^2)**

Q13. T(n) = 2T(n-1) + 1

Unrolling: $T(n) = 2T(n-1)+1 = 4T(n-2)+2+1 = 8T(n-3)+4+2+1 = ... = 2^nT(0) + (2^{n-1} + ... + 2 + 1) = 2^nT(0) + 2^n - 1$. This is exponential growth, doubling the recursive cost at each of n levels.

✓ **Answer: T(n) = Θ(2^n)**

Q14. T(n) = T(n/2) + 1

By the Master Theorem with a=1, b=2, f(n)=Θ(1): $n^{\log_b a} = n^0 = 1$, and f(n) = Θ(1) matches exactly — this is **Case 2** (with k=0), giving an extra log factor. Equivalently: unrolling gives log n levels each contributing constant work (binary search recurrence).

2

✓ **Answer: T(n) = Θ(log n)**

Q15. for(i=1; i<=n; i*=2) { for(j=1; j<=n; j++) { } }

```
for(int i=1;i<=n;i*=2)
{
for(int j=1;j<=n;j++)
{
}
}
```

The outer loop multiplies i by 2 each time starting from 1, running until i > n: this takes Θ(log n) iterations
(i = 1, 2, 4, 8, ..., up to n). For *each* of those outer iterations, the inner loop always runs the full n times (its bound n does not depend on i). So total work = (number of outer iterations) × (inner loop cost) = Θ(log n) × Θ(n).

✓ **Answer: Θ(n log n)**